TIENDA ONLINE API

Backend con Django, DRF, usuarios personalizados y JWT

Unidad 3 · Práctica 4
Desarrollo Web Integral

Proyecto: **tienda_online** — tema: tienda en línea.
Repositorio: https://github.com/Hector9229/django.git
Fecha: noviembre de 2026

Documento de entrega con las capturas de pantalla de Swagger, obtención del token, consulta de datos personales y la prueba de que editar un usuario sin contraseña no modifica su contraseña.

1. Datos del proyecto

Se tomó como base el proyecto de la Unidad 1 (Django + DRF + CRUD de productos) y se adaptó a los requisitos de la práctica. El proyecto se llama **tienda_online** y el tema es una tienda en línea; el título de Swagger es **Tienda Online API**.

Requisito	Implementación
Nombre y tema propios	tienda_online — tienda en línea
Título de Swagger	Tienda Online API
Campo extra del usuario	phone (teléfono), visible en admin, serializer y Swagger
Login	Email + contraseña (se eliminó username)
Token de acceso	ACCESS_TOKEN_LIFETIME = 60 minutos (configurable por variable de entorno)
Documentación	Swagger UI en /swagger/ y ReDoc en /redoc/
Base de datos	SQLite (db.sqlite3 excluido en .gitignore)

2. Endpoints principales

Método	Endpoint	Descripción	Permiso
POST	/api/v1/auth/token/	Obtener access y refresh	Público
POST	/api/v1/auth/token/refresh/	Renovar access	Público
GET	/api/v1/auth/me/	Datos personales	JWT
GET/POST	/api/v1/users/	Listar / crear usuarios	Admin
GET/PATCH/DELETE	/api/v1/users/{id}/	Detalle / editar / eliminar	Admin
GET/POST	/api/v1/products/	Listar / crear productos	JWT
GET/PATCH/DELETE	/api/v1/products/{id}/	Detalle / editar / eliminar	JWT
GET	/api/v1/products/{id}/precio-final/	Precio con descuento	JWT
GET/POST	/api/v1/categories/	Listar / crear categorías	JWT

En Swagger, para probar endpoints protegidos: ejecutar el login, copiar el valor de **access**, pulsar **Authorize** e ingresar **Bearer <access_token>**.

3. Correcciones de seguridad

El manual original incluía dos fallas de seguridad. Ambas fueron corregidas en el serializer de usuarios.

3.1 La contraseña se devolvía en las respuestas

El serializer incluía **password** como campo normal, por lo que el hash aparecía en el CRUD y en **/auth/me/**. Se declaró como **write_only=True**: se puede enviar al crear o actualizar, pero nunca se incluye en las respuestas.

```
password = serializers.CharField(
    write_only=True,
    required=False,
    allow_blank=False,
    min_length=8,
    style={'input_type': 'password'},
)
```

3.2 Editar sin contraseña cambiaba la contraseña

El manual asignaba **request.user.password** cuando la petición no traía contraseña. Eso cambiaba la contraseña del usuario editado por el hash del administrador que hacía la petición. Ahora la lógica está en el serializer y solo se llama a **set_password()** si la contraseña viene en la petición.

```
def update(self, instance, validated_data):
    password = validated_data.pop('password', None)

    for field, value in validated_data.items():
        setattr(instance, field, value)

    if password:
        instance.set_password(password)

    instance.save()
    return instance
```

3.3 Otras mejoras de seguridad

• Se eliminó **username**; el email es el identificador único.

• Las contraseñas se guardan con **set_password()** (hash de Django), nunca en texto plano.

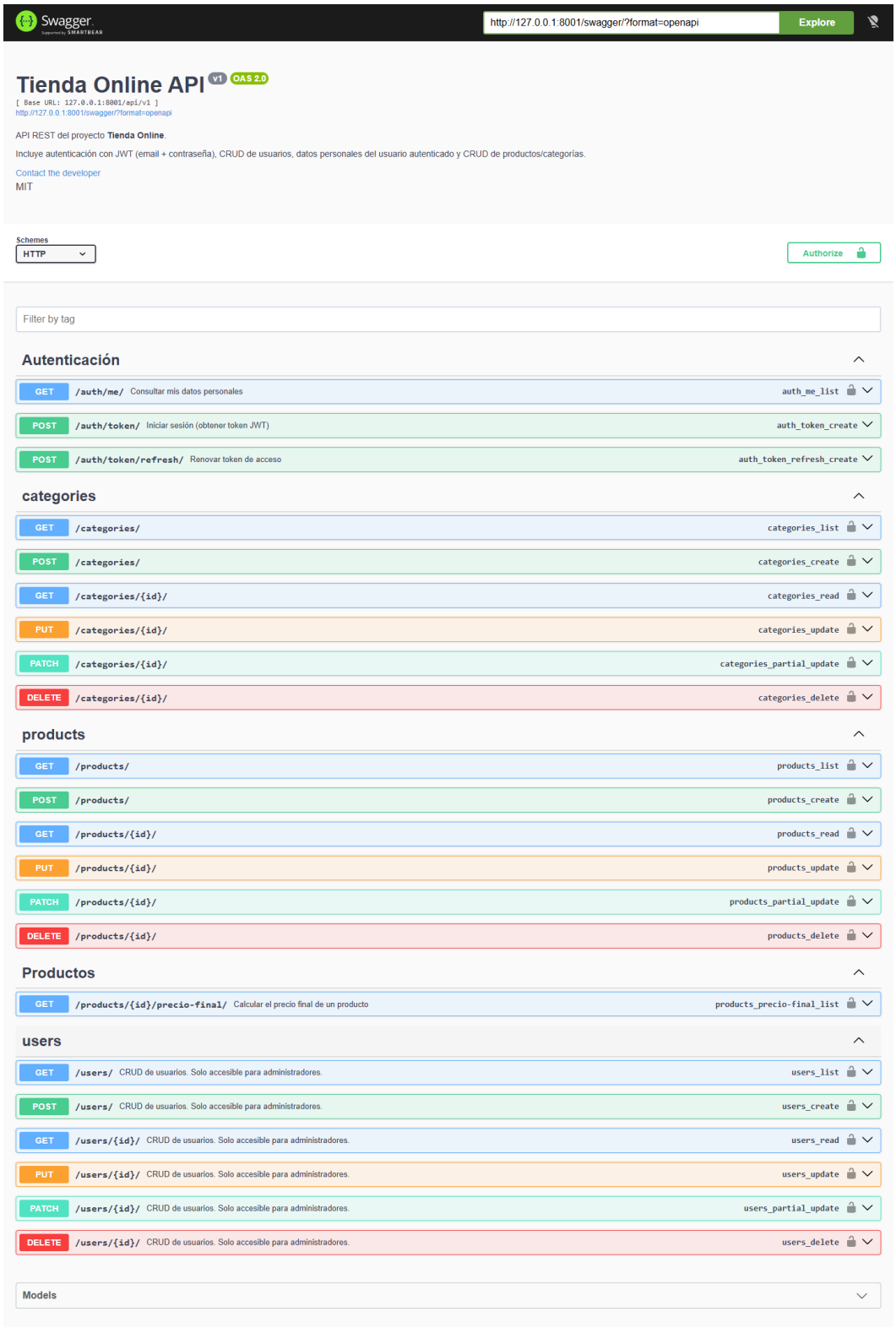
• Se validan con los validadores de contraseña de Django.

• El CRUD de usuarios requiere **IsAdminUser**; los endpoints de productos requieren JWT.

• El endpoint **precio_final** devuelve 404 si el producto no existe.

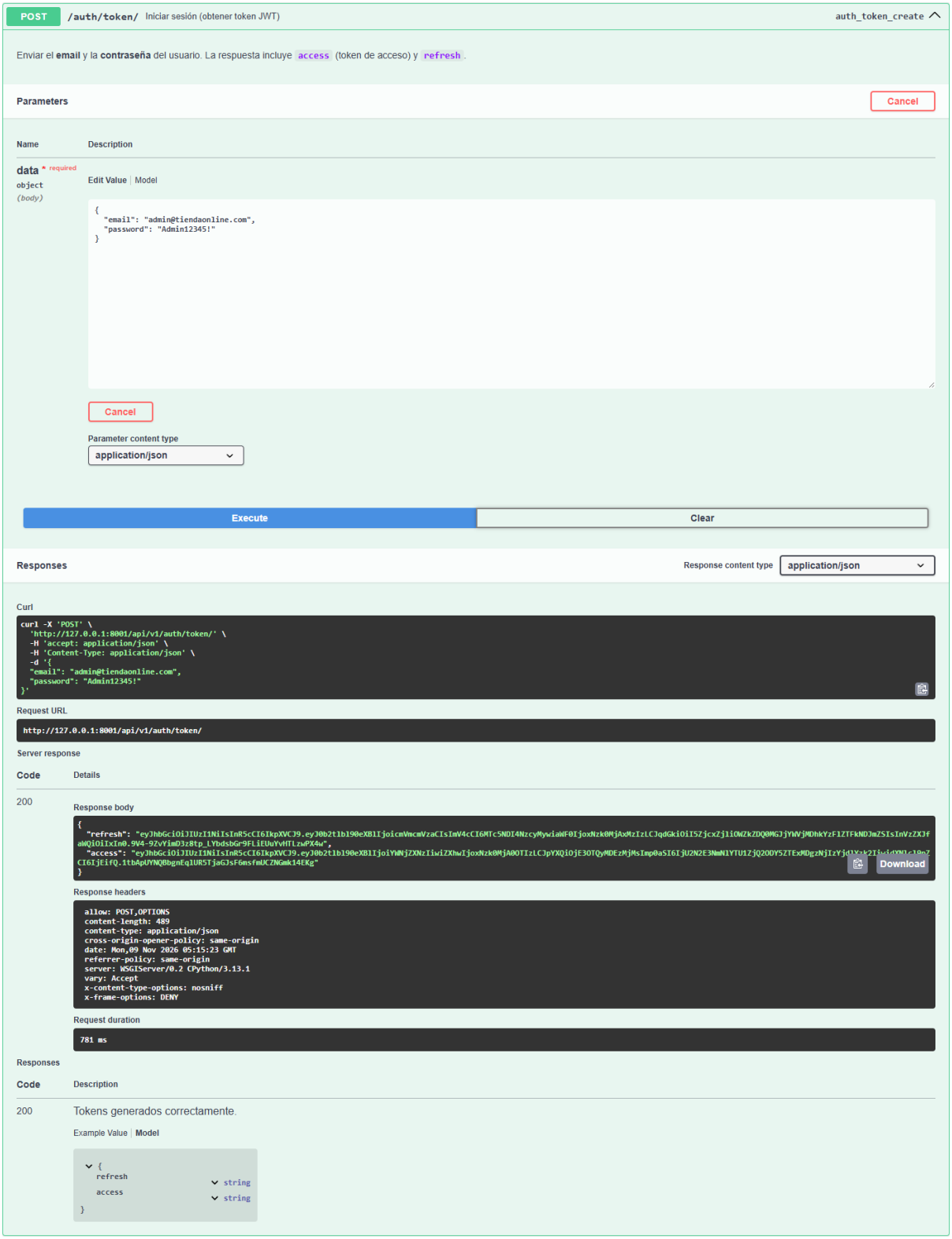
4. Capturas de pantalla

Las capturas se generaron contra **http://127.0.0.1:8001** porque el puerto 8000 estaba ocupado por otro proyecto en el equipo. Al ejecutar **python manage.py runserver** en un equipo libre, la URL es **http://127.0.0.1:8000**.



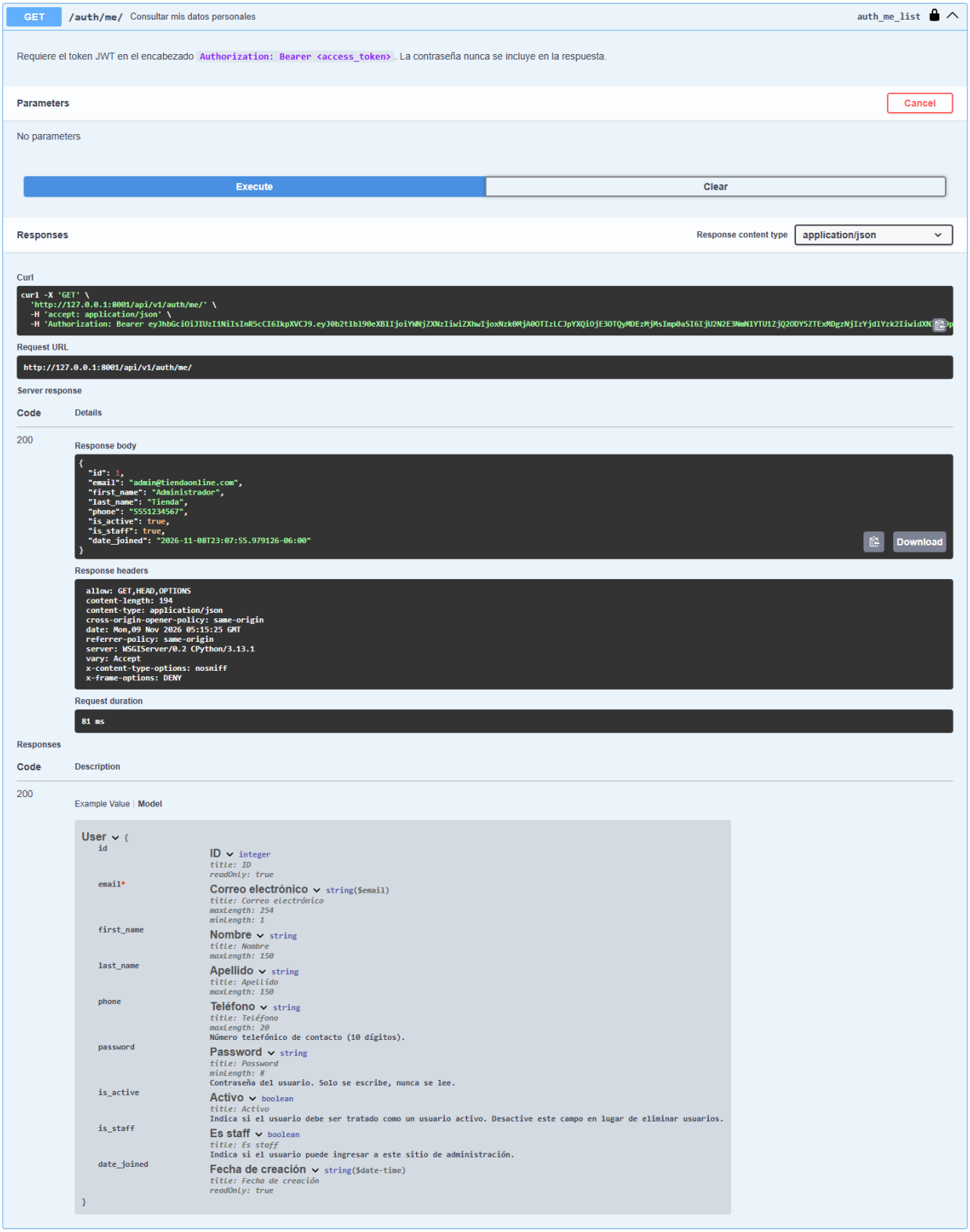
Captura 1. Swagger UI con todos los endpoints documentados. El título es **Tienda Online API** y se observan las secciones Autenticación, categories, products, Productos y users.

4. Capturas de pantalla (continuación)



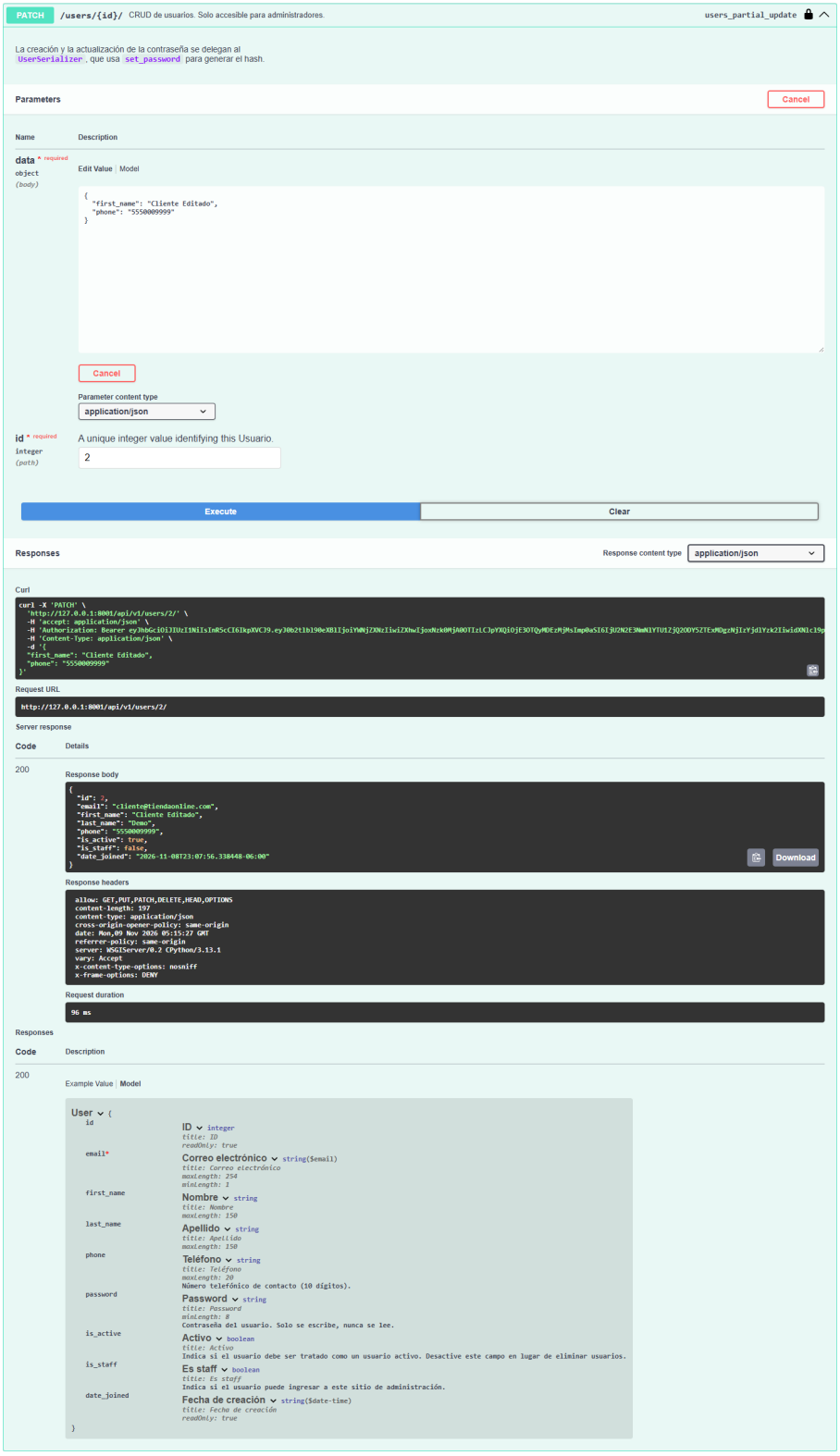
Captura 2. Obtención del token JWT con **POST /api/v1/auth/token/** usando email y contraseña. La respuesta incluye **refresh** y **access**.

4. Capturas de pantalla (continuación)

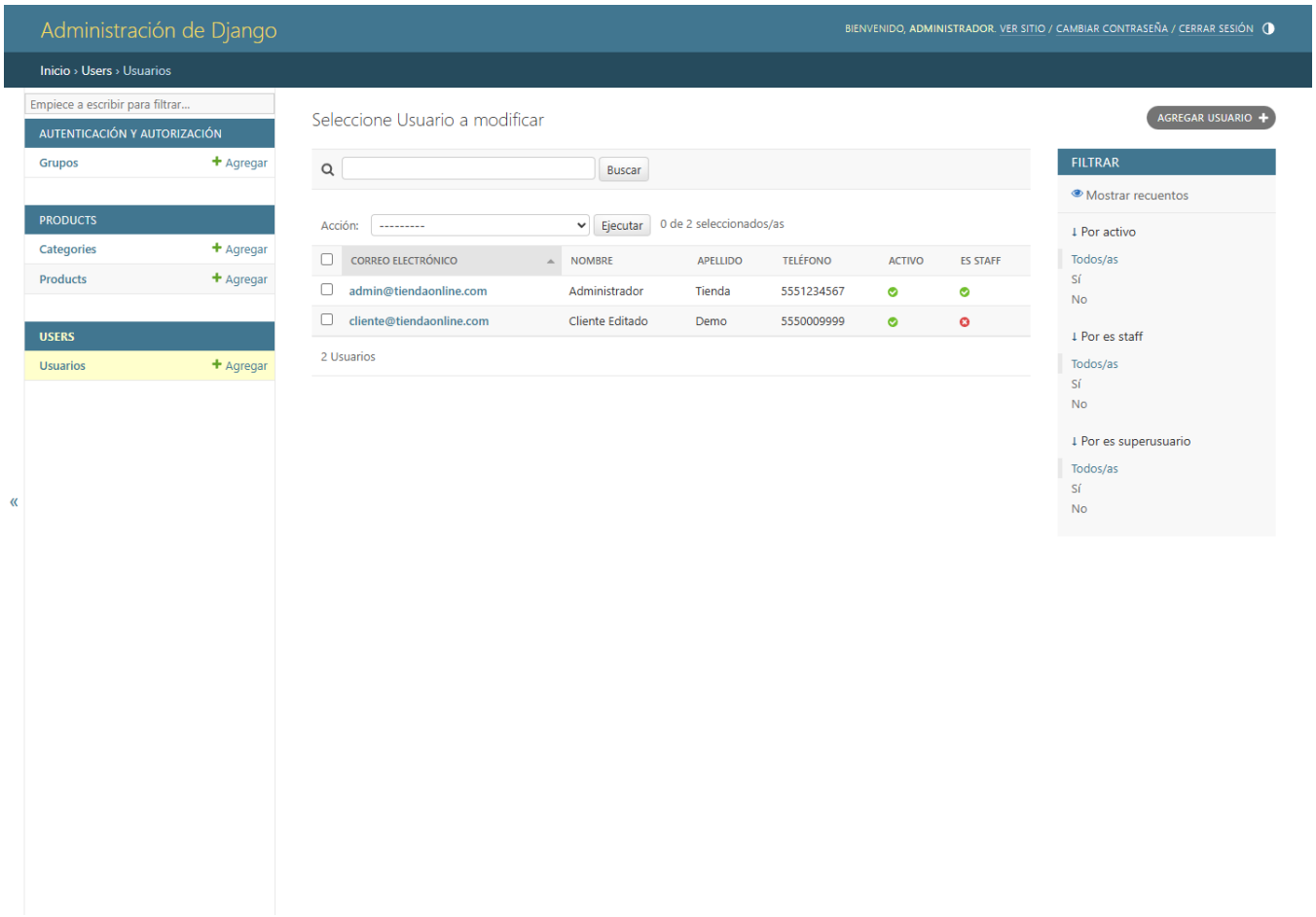


Captura 3. Consulta de datos personales con **GET /api/v1/auth/me/** usando el token en Authorize. La respuesta incluye el campo **phone** y **no incluye password**.

4. Capturas de pantalla (continuación)

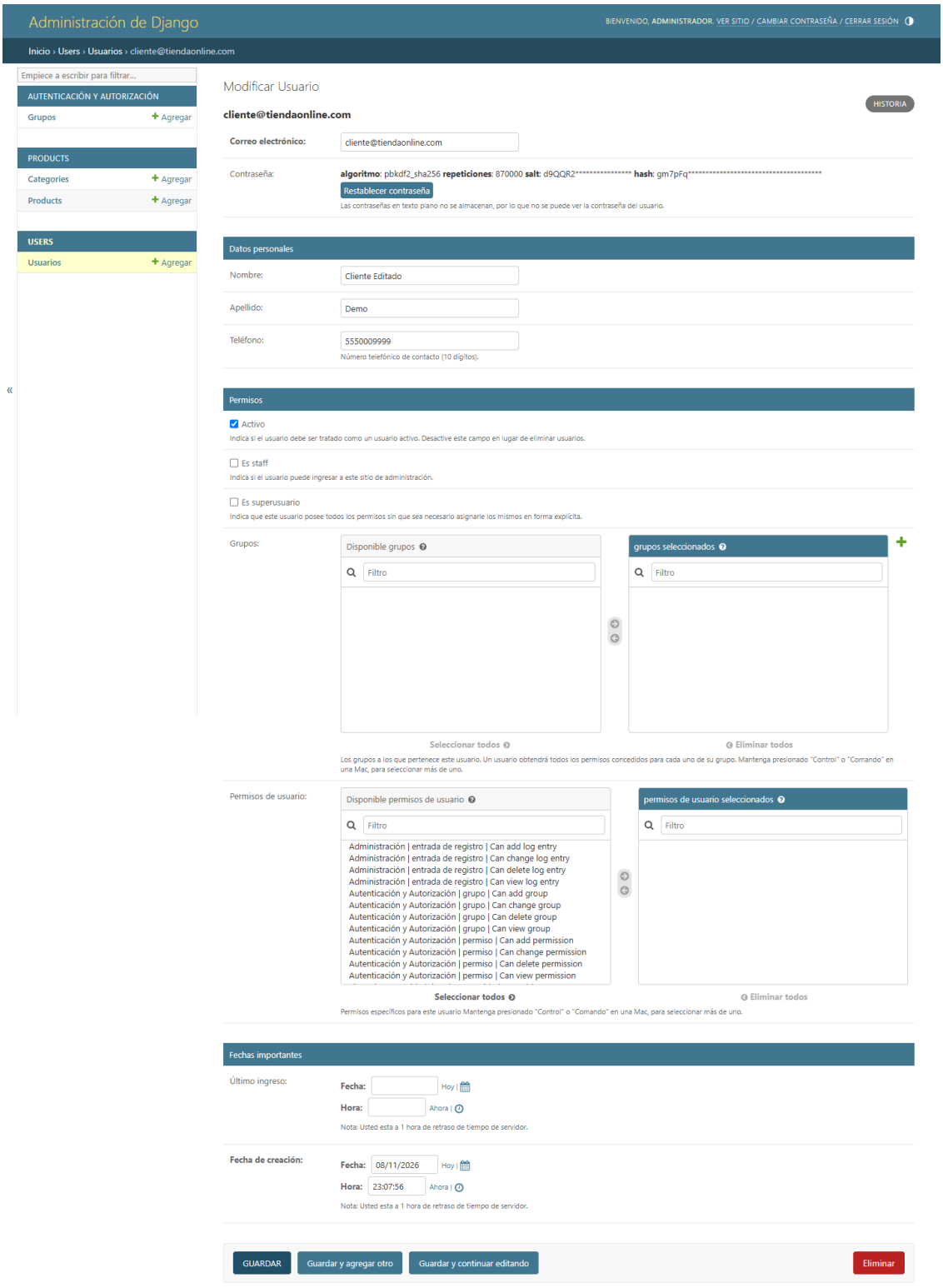

PATCH
/users/{id}/
CRUD de usuarios. Solo accesible para administradores.
users_partial_update
La creación y la actualización de la contraseña se delegan al UserSerializer, que usa set_password para generar el hash.
Parameters
Cancel
Name
Description
data * required
object
(body)
Edit Value | Model
{
"first_name": "Cliente Editado",
"phone": "5550009999"
}
Cancel
Parameter content type
application/json
id * required
integer
(path)
A unique integer value identifying this Usuario.
2
Execute
Clear
Responses
Response content type
application/json
Curl
curl -X 'PATCH' \
'http://127.0.0.1:8001/api/v1/users/2/' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90eXBlIjoiYWNjZXNzIiwiZXhwIjoxNzk0MjA0OTIzLCJpYXQiOjE3OTQyMDEzMjMsImp0aSI6IjU2N2E3MmM1YTU1ZjQ2ODY5ZTExMDgzNjIzYjdlYzk2IiwidXNlcl9pZCI6MX0.Ic19pMsJ0Ic19pMsJ0Ic19pMsJ0

4. Capturas de pantalla (continuación)



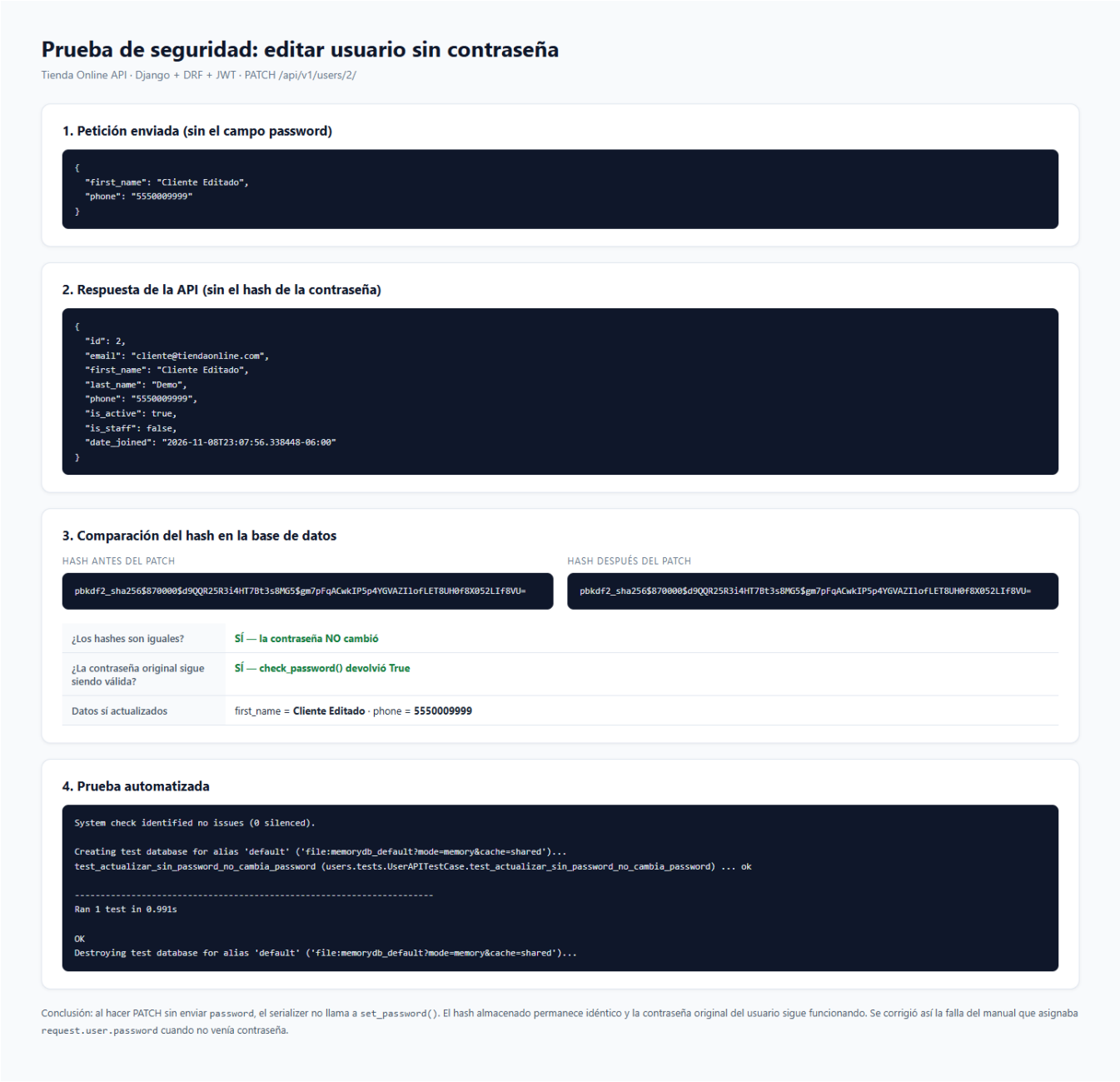
Captura 5. Admin de Django: listado de usuarios con la columna **Teléfono** (campo extra solicitado).

4. Capturas de pantalla (continuación)



Captura 6. Admin de Django: detalle del usuario con el campo **Teléfono** visible en el formulario.

4. Capturas de pantalla (continuación)



Prueba de seguridad: editar usuario sin contraseña

Tienda Online API · Django + DRF + JWT · PATCH /api/v1/users/2/

1. Petición enviada (sin el campo password)

```
{
  "first_name": "Cliente Editado",
  "phone": "5550009999"
}
```

2. Respuesta de la API (sin el hash de la contraseña)

```
{
  "id": 2,
  "email": "cliente@tiendaonline.com",
  "first_name": "Cliente Editado",
  "last_name": "Demo",
  "phone": "5550009999",
  "is_active": true,
  "is_staff": false,
  "date_joined": "2026-11-08T23:07:56.338448-06:00"
}
```

3. Comparación del hash en la base de datos

HASH ANTES DEL PATCH

```
pbkdf2_sha256$870000$d9QQR25R3i4HT7Bt3s8MG5$gm7pFqACwkIP5p4YGVAZI1ofLET8UH0f8X052LIf8VU=
```

HASH DESPUÉS DEL PATCH

```
pbkdf2_sha256$870000$d9QQR25R3i4HT7Bt3s8MG5$gm7pFqACwkIP5p4YGVAZI1ofLET8UH0f8X052LIf8VU=
```

¿Los hashes son iguales?	**SÍ — la contraseña NO cambió**
¿La contraseña original sigue siendo válida?	**SÍ — check_password() devolvió True**
Datos sí actualizados	first_name = **Cliente Editado** · phone = **5550009999**

4. Prueba automatizada

```
System check identified no issues (0 silenced).

Creating test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
test_actualizar_sin_password_no_cambia_password (users.tests.UserAPITestCase.test_actualizar_sin_password_no_cambia_password) ... ok

----------------------------------------------------------------------
Ran 1 test in 0.991s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...
```

Conclusión: al hacer PATCH sin enviar `password`, el serializer no llama a `set_password()`. El hash almacenado permanece idéntico y la contraseña original del usuario sigue funcionando. Se corrigió así la falla del manual que asignaba `request.user.password` cuando no venía contraseña.

Captura 7. Prueba de que editar sin contraseña no la cambia: el hash antes y después del PATCH es idéntico, **check_password()** devuelve **True** y la prueba automatizada termina en **OK**.

5. Repositorio y ejecución

Repositorio de GitHub: https://github.com/Hector9229/django.git

El repositorio incluye **.gitignore** (excluye **venv/** y **db.sqlite3**), **requirements.txt**, el proyecto Django y la documentación de Swagger.

Comandos para ejecutar

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver

# Swagger: http://127.0.0.1:8000/swagger/
# Admin:   http://127.0.0.1:8000/admin/
```

Usuarios de prueba

Rol	Email	Contraseña
Administrador	admin@tiendaonline.com	Admin12345!
Cliente	cliente@tiendaonline.com	Cliente12345!

Resultado de las pruebas automatizadas: **17 pruebas OK**. Entre ellas se verifica que el listado y /auth/me/ no exponen la contraseña, que crear un usuario guarda el hash, que editar sin contraseña conserva el hash y que editar con contraseña sí la cambia.